Dispositivos Conectados
Conectividade Wi-Fi

2026/2027

Authors: Paulo C. Bartolomeu

Versão 1.0
13 de outubro de 2026

Histórico de versões

Versão	Data	Alterações
1.0	2026-10-13	Versão inicial.

1 Objetivos

- Compreender a arquitetura de uma rede IEEE 802.11 e os modos de operação Wi-Fi do ESP32-C6;
- Compreender o modelo de programação Wi-Fi do ESP-IDF, baseado em eventos;
- Utilizar os modos *Station* (STA) e *SoftAP* (AP);
- Implementar e testar um servidor HTTPS no ESP32-C6, compreendendo o papel dos certificados TLS.

2 Introdução

2.1 O Wi-Fi no contexto dos *Dispositivos Conectados*

Um dispositivo conectado só tem valor se conseguir trocar dados com outros sistemas: enviar medições para um servidor, receber comandos de uma aplicação ou disponibilizar uma interface de configuração. A escolha da tecnologia de comunicação condiciona o consumo de energia, o alcance, o débito e a forma como o dispositivo se integra na rede. O Wi-Fi ocupa um lugar particular entre as tecnologias usadas em IoT: é a única que, de forma generalizada, liga o dispositivo **diretamente a uma rede IP** já existente (a de uma casa, de uma empresa ou do *hotspot* de um telemóvel), sem necessidade de um *gateway* dedicado. Um sensor ligado por Wi-Fi pode usar a mesma pilha de protocolos de qualquer computador (IP, TCP/UDP, HTTP, MQTT, TLS) e comunicar com serviços na *cloud* sem tradução intermédia.

Tecnicamente, o Wi-Fi corresponde à família de normas **IEEE 802.11**, que especifica a camada física (PHY) e a subcamada de controlo de acesso ao meio (MAC) de redes locais sem fios, ou seja, as camadas 1 e 2 do modelo OSI. Tudo o que está acima (IP, DHCP, TCP, HTTP) é independente

do meio físico e é implementado, no ESP32-C6, pela pilha lwIP. A designação "Wi-Fi" é uma marca da Wi-Fi Alliance, que certifica a interoperabilidade dos produtos e atribui nomes comerciais às sucessivas gerações da norma:

Tabela 2: Gerações da norma IEEE 802.11.

Norma	Designação comercial	Bandas	Aspetos relevantes
802.11b/g	(sem designação)	2,4 GHz	Base de compatibilidade, ainda suportada pela generalidade dos equipamentos
802.11n	Wi-Fi 4	2,4 e 5 GHz	MIMO e canais de 40 MHz
802.11ac	Wi-Fi 5	5 GHz	Canais mais largos e débitos mais elevados
802.11ax	Wi-Fi 6 / 6E	2,4, 5 e 6 GHz	OFDMA e *Target Wake Time* (TWT), pensados para redes com muitos dispositivos

O acesso ao meio é partilhado: antes de transmitir, cada estação escuta o canal e só transmite se o encontrar livre, aguardando um tempo aleatório para reduzir a probabilidade de colisões (CSMA/CA). Na banda dos 2,4 GHz, a Europa dispõe dos canais 1 a 13, espaçados de 5 MHz; como cada canal ocupa cerca de 20 MHz, canais próximos sobrepõem-se, e as redes vizinhas usam normalmente os canais 1, 6 e 11 para não interferirem entre si. A mesma banda é partilhada com o Bluetooth, o IEEE 802.15.4 (Zigbee, Thread) e os fornos micro-ondas, o que torna a coexistência e a escolha do canal aspetos práticos importantes.

O principal custo do Wi-Fi num dispositivo IoT é o **consumo de energia**. Para se manter associada, uma estação tem de receber periodicamente os *beacons* do AP, e a transmissão consome dezenas a centenas de miliamperes. O ESP-IDF ativa por omissão, em modo STA, uma poupança de energia mínima (`WIFI_PS_MIN_MODEM`): o rádio é desligado entre *beacons* e acorda em cada período DTIM, o intervalo com que o AP anuncia o tráfego retido para as estações adormecidas. O Wi-Fi 6 acrescenta o *Target Wake Time*, que permite ao dispositivo negociar com o AP os instantes em que acorda e permanecer adormecido durante longos períodos. Mesmo assim, para dispositivos alimentados a bateria durante anos, tecnologias como o BLE, o IEEE 802.15.4 ou o LoRa são muitas vezes mais adequadas:

Tabela 3: Comparação entre tecnologias sem fios usadas em IoT.

Tecnologia	Débito típico	Alcance típico	Consumo	Topologia	IP nativo
Wi-Fi (802.11)	dezenas a centenas de Mbit/s	dezenas de metros (interior)	elevado	estrela (AP)	sim
Bluetooth Low Energy	até 2 Mbit/s	dezenas de metros	muito baixo	ponto a ponto, estrela, malha	não

Tecnologia	Débito típico	Alcance típico	Consumo	Topologia	IP nativo
IEEE 802.15.4 (Zigbee, Thread)	250 kbit/s	dezenas de metros por salto	muito baixo	malha	Thread: sim (IPv6)
LoRa / LoRaWAN	até dezenas de kbit/s	quilómetros	muito baixo	estrela de estrelas (*gateways*)	não

O ESP32-C6 integra três destes rádios: Wi-Fi 6 (802.11b/g/n/ax) na banda dos 2,4 GHz, Bluetooth LE 5 e IEEE 802.15.4. Nesta aula usamos o Wi-Fi, primeiro como cliente de uma rede existente e depois criando a sua própria rede, e terminamos com um servidor HTTPS que expõe os dados de um sensor através dessa ligação.

2.2 Arquitetura de uma rede 802.11

O elemento fundamental de uma rede IEEE 802.11 é o *Basic Service Set* (BSS). Este pode apresentar duas categorias: *Independent BSS* e *Infrastructure BSS*. Num *Infrastructure BSS*, as estações (STAs) têm de se associar a um Ponto de Acesso (AP) para obter serviços de rede. Os AP funcionam como o centro de controlo do conjunto, sendo esta a arquitetura de rede mais comum. Cada STA deve passar pelos processos de autenticação e associação antes de entrar num determinado BSS. Ao associar-se, a STA recebe do AP um identificador numérico, o *Association ID* (AID).

Numa perspetiva prática, as redes Wi-Fi são acompanhadas por identificadores próprios:

- **BSS Identification (BSSID):** cada BSS possui um endereço físico de identificação, denominado BSSID. Num *Infrastructure BSS*, o BSSID corresponde ao endereço MAC da interface Wi-Fi do *Access Point* (AP). Este endereço é normalmente atribuído de fábrica, mas pode ser alterado por *software*: muitos telemóveis, quando funcionam como *hotspot*, geram um endereço MAC aleatório, que pode mudar quando o *hotspot* é reiniciado.
- **Service Set Identification (SSID):** cada AP dispõe também de um identificador destinado ao utilizador, denominado SSID. Na maioria dos casos, um BSSID está associado a um único SSID. Este identificador é normalmente um texto legível, sendo aquilo que vulgarmente chamamos de nome da rede Wi-Fi.

2.3 Wi-Fi no ESP32-C6

O ESP32-C6 suporta as normas IEEE 802.11b/g/n/ax (Wi-Fi 6) **apenas na banda dos 2,4 GHz**: não consegue ligar-se a redes de 5 GHz. A *framework* ESP-IDF permite que o SoC opere em três modos:

Tabela 4: Modos de operação Wi-Fi do ESP32-C6.

Modo	Constante	Descrição
Station	`WIFI_MODE_STA`	Liga-se a um AP existente e obtém um endereço IP por DHCP

Modo	Constante	Descrição
SoftAP	`WIFI_MODE_AP`	Cria a sua própria rede, anuncia o SSID e atribui endereços IP aos clientes por DHCP
Station + SoftAP	`WIFI_MODE_APSTA`	Funciona simultaneamente nos dois modos

Cada modo usa uma interface de rede própria, com o seu endereço MAC. Por omissão, os endereços são obtidos a partir de um endereço MAC base gravado no *chip*: a interface STA usa o endereço base e a interface SoftAP usa o endereço base com o último octeto incrementado de uma unidade.

Ao atuar como STA, o dispositivo pesquisa as redes disponíveis, autentica-se e associa-se a um BSSID; ao atuar como SoftAP, é o próprio ESP32-C6 que anuncia o SSID, fornece endereços IP por DHCP e gere as estações que se ligam. No modo SoftAP, o ESP32-C6 usa por omissão o endereço `192.168.4.1`, com máscara `255.255.255.0`, e atribui aos clientes endereços dessa sub-rede, a começar em `192.168.4.2`.

2.4 Modelo de programação Wi-Fi no ESP-IDF

As operações de rede do Wi-Fi no ESP-IDF são **assíncronas**: funções como `esp_wifi_connect()` apenas pedem a operação e retornam de imediato, sendo o resultado (ligação estabelecida, falha, endereço obtido) comunicado mais tarde através de **eventos**. A aplicação regista funções de tratamento (*event handlers*) no *event loop* por omissão, que as invoca quando o evento correspondente ocorre.

Os exemplos desta aula seguem a mesma sequência de inicialização:

1. `nvs_flash_init()`: inicializa a partição NVS da memória *flash*, onde o *driver* Wi-Fi guarda dados de configuração e calibração (opção `CONFIG_ESP_WIFI_NVS_ENABLED`, ativa por omissão);
2. `esp_netif_init()`: inicializa a pilha TCP/IP (lwIP) e a camada de abstração de interfaces de rede (`esp_netif`);
3. `esp_event_loop_create_default()`: cria o *event loop* por omissão;
4. `esp_netif_create_default_wifi_sta()` ou `esp_netif_create_default_wifi_ap()`: cria a interface de rede do modo pretendido (com cliente ou servidor DHCP, respetivamente);
5. `esp_wifi_init(&cfg)`, com `cfg = WIFI_INIT_CONFIG_DEFAULT()`: inicializa o *driver* Wi-Fi;
6. `esp_event_handler_instance_register(...)`: regista as funções de tratamento de eventos;
7. `esp_wifi_set_mode(...)` e `esp_wifi_set_config(...)`: definem o modo e a configuração (`wifi_config_t`);
8. `esp_wifi_start()`: arranca o Wi-Fi. No modo STA, a ligação ao AP é pedida com `esp_wifi_connect()`, normalmente quando ocorre o evento `WIFI_EVENT_STA_START`.

Os eventos mais relevantes nesta aula são os seguintes:

Tabela 5: Eventos Wi-Fi e IP utilizados nos exemplos.

Evento	Quando ocorre
`WIFI_EVENT_STA_START`	O Wi-Fi arrancou em modo STA
`WIFI_EVENT_STA_CONNECTED`	A STA associou-se ao AP (ainda sem endereço IP)
`WIFI_EVENT_STA_DISCONNECTED`	A STA perdeu a ligação ao AP ou falhou uma tentativa de ligação

Evento	Quando ocorre
`IP_EVENT_STA_GOT_IP`	A STA obteve um endereço IPv4 por DHCP
`WIFI_EVENT_AP_START`	O SoftAP arrancou
`WIFI_EVENT_AP_STACONNECTED`	Uma estação associou-se ao SoftAP (os dados do evento incluem o seu MAC e o AID)
`WIFI_EVENT_AP_STADISCONNECTED`	Uma estação desassociou-se do SoftAP
`IP_EVENT_ASSIGNED_IP_TO_CLIENT`	O servidor DHCP do SoftAP atribuiu um endereço a um cliente (substitui o antigo `IP_EVENT_AP_STAIPASSIGNED`, marcado como obsoleto no ESP-IDF v6.1)

Como as funções de tratamento de eventos correm na tarefa do *event loop*, os exemplos usam um *event group* do FreeRTOS para que `app_main()` possa esperar pelo resultado: a função de tratamento de eventos ativa um bit (por exemplo, `WIFI_CONNECTED_BIT` ou `WIFI_FAIL_BIT`) e a tarefa principal fica bloqueada em `xEventGroupWaitBits()` até que um deles seja ativado.

A Figura 1 resume a sequência completa no modo STA, tal como implementada no exemplo `station`: a inicialização pela tarefa principal, a ligação ao AP desencadeada pelo evento `WIFI_EVENT_STA_START`, a obtenção do endereço IP por DHCP e o tratamento de uma perda de ligação.

2.5 Segurança

O campo `authmode` de `wifi_config_t` define o modo de autenticação: `WIFI_AUTH_OPEN` (rede aberta), `WIFI_AUTH_WPA2_PSK`, `WIFI_AUTH_WPA3_PSK` ou `WIFI_AUTH_WPA2_WPA3_PSK`, entre outros. O modo WPA2-PSK exige uma *password* com 8 a 63 caracteres.

- Em modo **STA**, o campo `threshold.authmode` define o modo de autenticação **mínimo** que a STA aceita: o ESP32-C6 não se liga a redes com segurança inferior a esse limiar. No exemplo `station`, o limiar é configurável na opção `WiFi Scan auth mode threshold` e é, por omissão, WPA2-PSK.
- Em modo **SoftAP**, o campo `authmode` define o modo usado pela rede criada. No exemplo `softAP` do ESP-IDF v6.1, com a opção `Enable WPA3 Personal(SAE) SoftAP` ativa (o que acontece por omissão), a rede usa **apenas WPA3-PSK**, com *Protected Management Frames* (PMF) obrigatórios. Clientes sem suporte de WPA3 não se conseguem ligar. Se a *password* ficar vazia, a rede é criada aberta (`WIFI_AUTH_OPEN`).

3 Trabalho Prático

O trabalho prático desta aula desenvolve-se com base na API de Referência Wi-Fi do ESP32-C6, no Guia do *driver* Wi-Fi e na API de Referência HTTPS Server do ESP32-C6, juntamente com o ESP32-C6 Technical Reference Manual.

Versão do ESP-IDF: todos os exercícios devem ser realizados com o ESP-IDF **v6.1.0**. Noutras versões, o código dos exemplos, as opções de configuração e as mensagens apresentadas na consola podem diferir.

Preparação do *hotspot* do telemóvel:

- O ESP32-C6 só funciona na banda dos 2,4 GHz. No iPhone, ative a opção **Maximizar compatibilidade** do *hotspot*; noutros telemóveis, escolha a banda de 2,4 GHz.

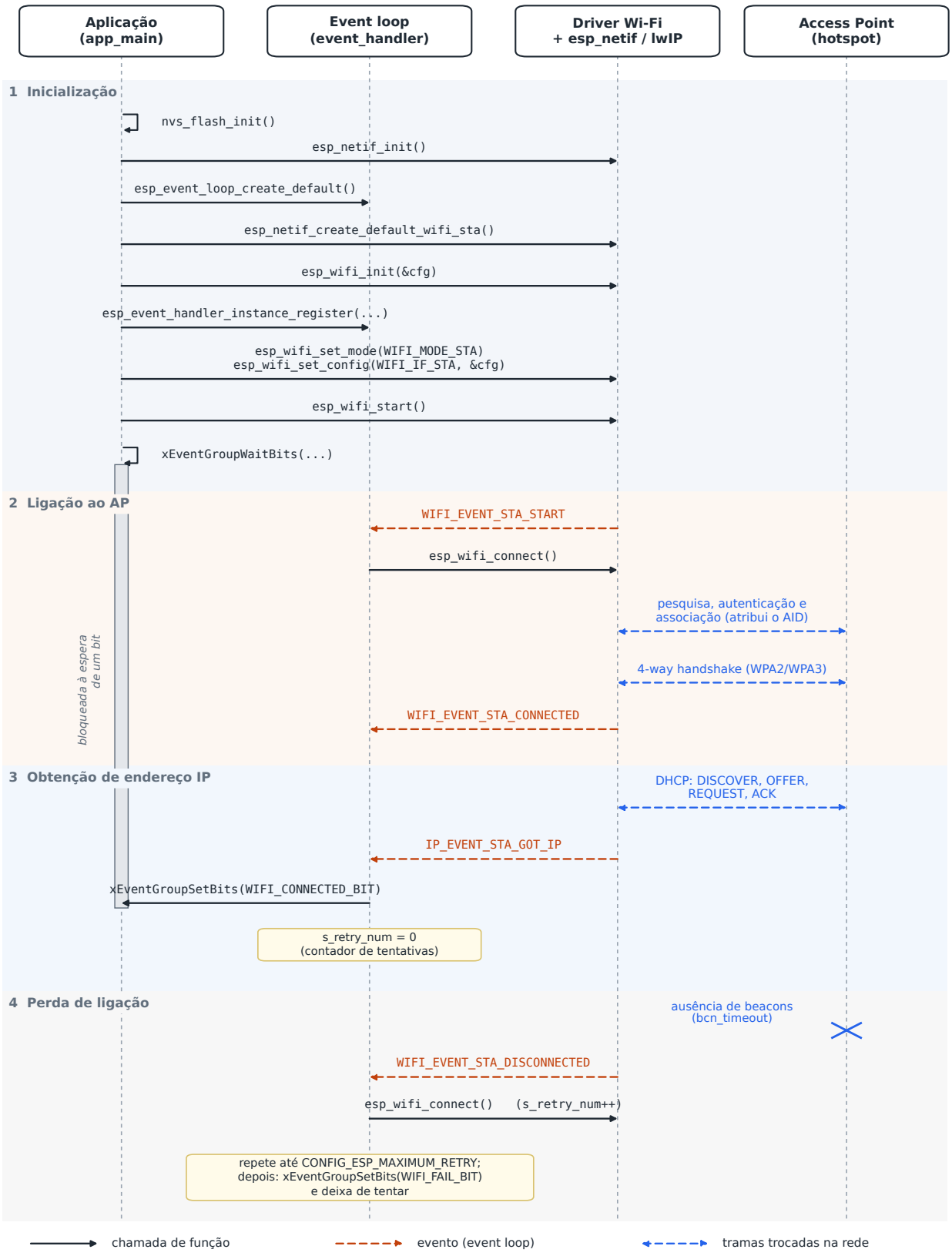


Figura 1: Sequência de inicialização e ligação Wi-Fi em modo STA no ESP-IDF

- Use segurança WPA2 (ou WPA2/WPA3) e uma *password* com, pelo menos, 8 caracteres.
- Redes institucionais como a *eduroam* usam autenticação empresarial (WPA2-Enterprise), que os exemplos desta aula não suportam.

3.1 Wi-Fi Modo Cliente (STA)

O exemplo do Modo Cliente Wi-Fi demonstra como configurar o ESP32-C6 para funcionar no modo *Station* (STA), ligando-se a um ponto de acesso Wi-Fi existente, como por exemplo um *Access Point* (AP) criado pelo telemóvel do aluno. O programa inicia o subsistema de rede, tenta ligar-se à rede definida no `menuconfig` e regista eventos para acompanhar o processo de ligação.

A função de tratamento de eventos do exemplo funciona da seguinte forma:

- em `WIFI_EVENT_STA_START`, chama `esp_wifi_connect();`
- em `WIFI_EVENT_STA_DISCONNECTED`, se o número de tentativas for inferior a `CONFIG_ESP_MAXIMUM_RETRY` (opção `Maximum retry`, 5 por omissão), volta a chamar `esp_wifi_connect()`, incrementa o contador e imprime `retry to connect to the AP`; caso contrário, ativa `WIFI_FAIL_BIT` **e deixa de tentar**. Em ambos os casos imprime `connect to the AP fail`;
- em `IP_EVENT_STA_GOT_IP`, imprime o endereço obtido, **repõe o contador de tentativas a zero** e ativa `WIFI_CONNECTED_BIT`.

Quando a ligação é bem-sucedida, o dispositivo obtém um endereço IPv4 via DHCP do AP, e essa informação é apresentada no terminal.

> **QUESTÃO 1:** No VSCode crie um novo projeto a partir do exemplo `wifi -> getting_started -> station`. No `SDK Configuration Editor` (`menuconfig`), na secção `Example Configuration`, defina o `WiFi SSID` e a `WiFi Password` da rede Wi-Fi que tiver configurado no seu telemóvel. Mantenha as restantes opções com os valores por omissão. Compile o programa e descarregue o *firmware* para o SoC. Deverá observar um *log* de eventos semelhante ao seguinte (os endereços, o SSID, o canal e os tempos dependem da sua rede):

```
...
I (346) main_task: Started on CPU0
I (356) main_task: Calling app_main()
I (376) wifi station: ESP_WIFI_MODE_STA
...
I (566) wifi:mode : sta (58:8c:81:3b:c0:bc)
...
I (866) wifi:connected with phobar, aid = 1, channel 6, BW20, bssid = 36:14:26:eb:1f:77
I (13586) wifi:security: WPA2-PSK, phy:ax, rssi:-37, cipher(pairwise:0x3, group:0x3), pmf:0,
...
I (14966) esp_netif_handlers: sta ip: 172.20.10.2, mask: 255.255.255.240, gw: 172.20.10.1
I (14966) wifi station: got ip:172.20.10.2
```

> Identifique no *log*: o endereço MAC da interface STA do ESP32-C6, o BSSID do AP, o AID atribuído, o canal, o modo de segurança, a norma física utilizada (`phy`) e o endereço IP, a máscara e o *gateway* obtidos por DHCP.

> **QUESTÃO 2:** Agora que o ESP32-C6 possui conectividade IPv4, ligue o PC ao mesmo *Access Point* e teste a conectividade com o dispositivo IoT através do comando `ping <IP do ESP32-C6>` (no Linux e no macOS, use `ping -c 10 <IP>` para enviar 10 pacotes). O teste deverá ser bem-sucedido, com latências relativamente reduzidas, de poucas

unidades/dezenas de milissegundos, face ao reduzido número de *hops* na ligação entre os dois *peers*. A pilha lwIP do ESP32-C6 responde aos pedidos ICMP *echo* sem qualquer código adicional na aplicação.

Se o `ping` falhar, verifique se o PC está ligado à mesma rede e se o AP não isola os clientes entre si (alguns pontos de acesso impedem a comunicação direta entre dispositivos ligados à mesma rede).

QUESTÃO 3: Cause uma disrupção momentânea no AP, por exemplo desligando o *hotspot* durante 2 a 3 segundos, e volte a ativá-lo. Deverá observar um comportamento consistente com a seguinte listagem:

```
I (88156) wifi:bcn_timeout,ap_probe_send_start
I (90656) wifi:ap_probe_send over, reset wifi status to disassoc
I (90656) wifi:state: run -> init (0xc800)
...
I (90676) wifi station: retry to connect to the AP
I (90676) wifi station: connect to the AP fail
I (93086) wifi station: retry to connect to the AP
I (93086) wifi station: connect to the AP fail
...
I (93496) wifi:connected with phobar, aid = 1, channel 6, BW20, bssid = 52:6a:a3:22:c6:39
I (93506) wifi:security: WPA2-PSK, phy:ax, rssi:-34, cipher(pairwise:0x3, group:0x3), pmf:0,
...
I (94836) esp_netif_handlers: sta ip: 172.20.10.2, mask: 255.255.255.240, gw: 172.20.10.1
I (94836) wifi station: got ip:172.20.10.2
```

a) Explique a sequência de mensagens: como é que o ESP32-C6 deteta a perda do AP (`bcn_timeout`, ausência de *beacons*), que evento é gerado e como é que a função de tratamento de eventos reage.

b) Compare o BSSID antes e depois da disrupção. Porque é que é diferente, se o SSID é o mesmo?

c) Desligue agora o *hotspot* durante cerca de 30 segundos e volte a ativá-lo. O ESP32-C6 volta a ligar-se? Explique o resultado com base no código do exemplo e na opção `Maximum retry`. Que alteração ao código faria para que o dispositivo tentasse ligar-se indefinidamente, mas sem sobrecarregar o AP com tentativas sucessivas?

Na disrupção curta, a ligação é restabelecida e o endereço IP é obtido novamente.

3.2 Wi-Fi Modo "Soft" *Access Point* (SoftAP)

No exemplo SoftAP vamos configurar o ESP32-C6 para operar como um *Access Point* Wi-Fi. Isto significa que o próprio ESP32-C6 cria e anuncia uma rede Wi-Fi autónoma, permitindo que outros dispositivos, como *smartphones*, computadores portáteis ou módulos IoT, se liguem diretamente a ele, sem necessidade de um *router* externo.

A rede é configurada no `menuconfig`, na secção `Example Configuration`, com as seguintes opções:

Tabela 6: Configuração do exemplo SoftAP.

Opção	Campo de `wifi_config_t.ap`	Valor por omissão
`WiFi SSID`	`ssid`	`myssid`
`WiFi Password`	`password`	`mypassword`

Opção	Campo de `wifi_config_t.ap`	Valor por omissão
`WiFi Channel`	`channel`	1 (gama 1 a 13)
`Maximal STA connections`	`max_connection`	4 (máximo 15 no ESP32-C6)

O exemplo inclui uma função de tratamento de eventos que imprime mensagens de *log* sempre que uma estação se liga (`WIFI_EVENT_AP_STACONNECTED`) ou desliga (`WIFI_EVENT_AP_STADISCONNECTED`) do ponto de acesso, indicando o endereço MAC do dispositivo e o identificador de associação (AID). Desta forma, é possível saber exatamente quando e quem entrou na rede. O servidor DHCP do SoftAP regista também, na consola, o endereço atribuído a cada cliente.

> **IMPORTANTE:** como referido na secção sobre segurança, no ESP-IDF v6.1 o exemplo cria, por omissão, uma rede **apenas WPA3-PSK**. Se o seu portátil não suportar WPA3 e não conseguir ligar-se, desative a opção `Component config > Wi-Fi > Enable WPA3 Personal(SAE) SoftAP` no `menuconfig`; a rede passa a usar WPA2-PSK.

> **QUESTÃO 4:** No VSCode crie um novo projeto a partir do exemplo `wifi -> getting_started -> softAP`. No `menuconfig`, na secção `Example Configuration`, defina o `WiFi SSID` e a `WiFi Password` (com, pelo menos, 8 caracteres) da rede a criar. Se todos os grupos usarem os valores *default* todas as placas com o exemplo softAP criam a rede **myssid**, com a mesma **password** e no **canal 1**. De modo a evitar que os grupos se liguem a APs de outros, cada um deve estabelecer um SSID próprio e escolher um canal aleatório entre os canais 1, 6 e 11. Compile o programa e descarregue o *firmware* para o SoC. Deverá observar um *log* de eventos semelhante ao seguinte:

```
I (356) main_task: Started on CPU0
I (366) main_task: Calling app_main()
I (386) wifi softAP: ESP_WIFI_MODE_AP
...
I (576) wifi:mode : softAP (58:8c:81:3b:c0:bd)
...
I (626) esp_netif_lwip: DHCP server started on interface WIFI_AP_DEF with IP: 192.168.4.1
I (636) wifi softAP: wifi_init_softap finished. SSID:<selected SSID> password:<selected password>
    channel:<selected channel>
I (646) main_task: Returned from app_main()
```

> Compare o endereço MAC da interface SoftAP com o endereço MAC da interface STA obtido na Questão 1, na mesma placa. Que relação existe entre eles?

Nesta fase, o ESP32-C6 opera como um *Access Point*, estando pronto para receber ligações e atribuir endereços (*leases*) DHCP.

> **QUESTÃO 5:** Usando o portátil, selecione a rede Wi-Fi criada pelo ESP32-C6 e ligue-se com a *password* definida. Deverá observar na consola mensagens de *log* com a indicação da estação que se associou e do endereço IP que lhe foi atribuído, tal como na listagem seguinte:

```
I (32756) wifi:station: e2:86:0b:c1:08:a0 join, AID=1, bgn, 20
I (32776) wifi softAP: station e2:86:0b:c1:08:a0 join, AID=1
I (33986) esp_netif_lwip: DHCP server assigned IP to a client, IP is: 192.168.4.2
```

Desligue o portátil da rede e observe a mensagem `station ... leave, AID=..., reason=...`. Ligue um segundo dispositivo (por exemplo, o telemóvel de um colega) e verifique o AID e o endereço IP que lhe são atribuídos.

QUESTÃO 6: Utilizando o comando `ping 192.168.4.1`, teste a conectividade entre o portátil e o *Access Point*. Deverá observar um resultado semelhante ao da Figura 2.

```
bartolomeu@b-dell-ubu-22:~/Downloads$ ping 192.168.4.1
PING 192.168.4.1 (192.168.4.1) 56(84) bytes of data.
64 bytes from 192.168.4.1: icmp_seq=1 ttl=64 time=2.31 ms
64 bytes from 192.168.4.1: icmp_seq=2 ttl=64 time=2.73 ms
64 bytes from 192.168.4.1: icmp_seq=3 ttl=64 time=4.88 ms
64 bytes from 192.168.4.1: icmp_seq=4 ttl=64 time=2.62 ms
64 bytes from 192.168.4.1: icmp_seq=5 ttl=64 time=2.67 ms
^C
--- 192.168.4.1 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4005ms
rtt min/avg/max/mdev = 2.305/3.040/4.877/0.929 ms
```

Figura 2: Teste de conectividade ao *Access Point*

O modo SoftAP é útil em várias fases do ciclo de vida de um dispositivo IoT. Em primeiro lugar, facilita a configuração inicial, permitindo que o utilizador se ligue diretamente ao dispositivo através do telemóvel para introduzir as credenciais da rede Wi-Fi ou ajustar parâmetros essenciais. Este modo é igualmente vantajoso em cenários onde o dispositivo precisa de funcionar de forma autónoma, sem depender de um *router* externo, como em instalações temporárias, ambientes industriais isolados ou medições de campo. Além disso, pode ser uma ferramenta muito prática para diagnóstico e manutenção, possibilitando recuperar dispositivos com falhas de ligação, alterar configurações ou realizar testes sem recorrer a cabos ou infraestruturas adicionais. Por fim, o SoftAP permite disponibilizar interfaces de controlo locais, como páginas web de configuração ou *dashboards*, assegurando comunicação direta sem necessidade de acesso à Internet.

3.3 Servidor Web HTTPS

3.3.1 HTTPS, TLS e certificados

O HTTPS corresponde ao protocolo HTTP transportado sobre uma ligação TLS (*Transport Layer Security*), que cifra a comunicação e permite ao cliente verificar a identidade do servidor. Por omissão, o HTTP usa a porta TCP 80 e o HTTPS a porta TCP 443.

No estabelecimento da ligação TLS (*handshake*), o servidor apresenta um **certificado**, que associa a sua chave pública a uma identidade. O *browser* só aceita o certificado sem avisos se se verificarem duas condições:

- o certificado foi emitido (assinado) por uma autoridade de certificação (CA) em que o sistema confia, como a Let's Encrypt ou a DigiCert;
- o endereço usado para aceder ao servidor (nome ou endereço IP) consta da extensão *Subject Alternative Name* (SAN) do certificado.

Um certificado *self-signed*, ou assinado por uma CA criada para testes, não cumpre a primeira condição; o *browser* apresenta um aviso de segurança, que em desenvolvimento pode ser aceite como exceção.

O TLS permite também que o **servidor verifique a identidade do cliente**, pedindo-lhe um certificado próprio (autenticação mútua). Esta funcionalidade é usada, por exemplo, entre dispositivos

IoT e servidores na *cloud*, mas os *browsers* normalmente não dispõem de certificados de cliente.

3.3.2 O exemplo `https_server/simple`

Nesta secção vamos abordar o exemplo `https_server/simple` do ESP-IDF para o ESP32-C6. Este exemplo demonstra como criar um servidor web seguro (HTTPS) a correr diretamente no dispositivo:

- a ligação à rede é feita pela função auxiliar `example_connect()` (componente `protocol_examples_common`), que liga o ESP32-C6 em modo STA e espera pela obtenção dos endereços IPv4 e IPv6 *link-local*;
- o certificado do servidor (`servercert.pem`), a respetiva chave privada (`prvtkey.pem`) e o certificado da CA (`cacert.pem`) estão na pasta `main/certs` e são embebidos no *firmware* durante a compilação (diretiva `EMBED_TXTFILES` em `main/CMakeLists.txt`);
- o servidor é iniciado com `httpd_ssl_start()`, escuta na porta 443 e regista um *handler* para o método GET na rota `/`, que responde com uma pequena página HTML;
- o servidor é iniciado no evento `IP_EVENT_STA_GOT_IP` e parado no evento `WIFI_EVENT_STA_DISCONNECTED`, acompanhando o estado da ligação Wi-Fi.

No ESP-IDF v6.1, o exemplo configura o servidor para **exigir um certificado de cliente**, assinado pela CA de `cacert.pem` (autenticação mútua). Um *browser* sem esse certificado não consegue concluir o *handshake* TLS e não chega a mostrar a página. Para aceder ao servidor a partir do *browser*, é necessário ativar a opção `Example Configuration > Skip client certificate`: o servidor continua a pedir um certificado de cliente, mas aceita a ligação sem ele.

O certificado do servidor incluído no exemplo foi emitido para os nomes `esp32-server` e `localhost`, e não para o endereço IP do ESP32-C6. Acedendo pelo endereço IP, o *browser* apresenta um aviso de segurança, que deve ser aceite como exceção. O `README.md` do exemplo descreve como gerar um certificado com o endereço IP na extensão SAN, o que será necessário na Questão 10.

Para este exercício, **precisará de configurar o seu telemóvel como *Access Point*, de modo a que este atribua um endereço IP ao ESP32-C6 quando arrancar**.

> **QUESTÃO 7:** No VSCode crie um novo projeto a partir do exemplo `protocols -> https_server -> simple`. No `menuconfig`:
>
> - na secção `Example Connection Configuration` (e não `Example Configuration`), defina o `WiFi SSID` e a `WiFi Password` da rede criada pelo seu telemóvel;
> - na secção `Example Configuration`, ative a opção `Skip client certificate`.
>
> Compile o programa e descarregue o *firmware* para o SoC. Deverá observar um *log* de eventos semelhante ao seguinte:

```
...
I (23) boot: ESP-IDF v6.1 2nd stage bootloader
...
I (403) main_task: Started on CPU0
I (403) main_task: Calling app_main()
I (413) example_connect: Start example_connect.
...
I (603) wifi:mode : sta (58:8c:81:3b:c0:bc)
...
I (613) example_connect: Connecting to phobar...
W (623) wifi:Password length matches WPA2 standards, authmode threshold changes from OPEN to WPA2
I (633) example_connect: Waiting for IP(s)
...
I (4413) example_connect: Got IPv6 event: Interface "example_netif_sta" address:
    fe80:0000:0000:0000:5a8c:81ff:fe3b:c0bc, type: ESP_IP6_ADDR_IS_LINK_LOCAL
I (4523) example: Starting server
```

```
I (4523) esp_https_server: Starting server
I (4523) esp_https_server: Server listening on port 443
I (4523) example: Registering URI handlers
I (4523) esp_netif_handlers: example_netif_sta ip: 172.20.10.2, mask: 255.255.255.240, gw: 172.20.10.1
I (4533) example_connect: Got IPv4 event: Interface "example_netif_sta" address: 172.20.10.2
I (4543) example_common: Connected to example_netif_sta
I (4543) example_common: - IPv4 address: 172.20.10.2,
I (4553) example_common: - IPv6 address: fe80:0000:0000:0000:5a8c:81ff:fe3b:c0bc, type:
    ESP_IP6_ADDR_IS_LINK_LOCAL
I (4563) main_task: Returned from app_main()
```

Neste momento, o servidor está ativo, tendo-se ligado ao *Access Point* criado pelo telemóvel. O AP atribuiu o endereço IP 172.20.10.2 ao ESP32-C6, que serve uma página web nesse endereço, **na porta 443 (HTTPS)**. Dependendo da configuração do seu AP, o endereço IP atribuído poderá ser diferente.

> **QUESTÃO 8:** Ligue o seu computador ao *Access Point* criado pelo telemóvel e aceda ao endereço `https://172.20.10.2/` (substitua pelo endereço do seu ESP32-C6). É necessário indicar `https://`: sem ele, o *browser* tenta normalmente `http://`, na porta 80, onde o ESP32-C6 não responde. O *browser* irá apresentar um aviso de segurança, impedindo-o de visualizar imediatamente a página servida pelo ESP32-C6.

O aviso surge por duas razões. Em primeiro lugar, o certificado do servidor foi assinado por uma CA de teste incluída no exemplo, e não por uma autoridade de certificação reconhecida; o *browser* não tem forma de confirmar a identidade do servidor e considera a ligação potencialmente insegura. Em segundo lugar, o certificado foi emitido para os nomes `esp32-server` e `localhost`, que não correspondem ao endereço usado (`172.20.10.2`). Em contexto de desenvolvimento e de testes, este comportamento é esperado. As soluções consistem em aceitar a exceção de segurança no *browser*, gerar um certificado com o endereço correto na extensão SAN ou instalar a CA emissora como confiável no sistema.

Por agora, aceite a exceção de segurança. O *browser* pode ainda perguntar que certificado de cliente pretende usar, uma vez que o servidor o pede; nesse caso, prossiga sem certificado. Deverá observar a página web da Figura 3.

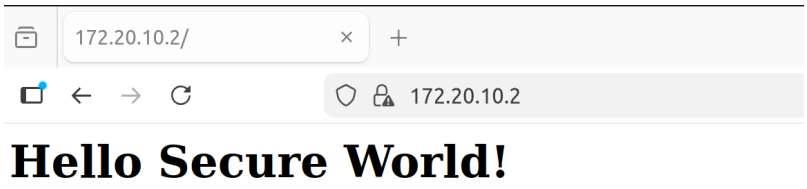


Figura 3: Página obtida do servidor web básico

> **QUESTÃO 9:** Altere o programa de modo a que a página servida na rota `/` apresente a temperatura e a humidade instantâneas, medidas pelo sensor DHT20 em cada pedido. Requisitos:
>
> - o DHT20 é ligado como na aula anterior (Tabela 7) e acedido com o *driver* `driver/i2c_master.h`. O *driver* legado `driver/i2c.h` está em fim de vida desde o ESP-IDF v6.0: compila com um aviso e será removido na versão 7.0. Os dois *drivers* não podem coexistir no mesmo programa;

- a medição é feita no *handler* da rota `/`, que responde com uma página HTML (`httpd_resp_set_type(req, "text/html")` e `httpd_resp_send()`) contendo os dois valores com uma casa decimal. Note que cada medição bloqueia o *handler* durante cerca de 80 ms;
- se a leitura do sensor falhar, o *handler* responde com o erro HTTP 500 (`httpd_resp_send_err(req, HTTPD_500_INTERNAL_SERVER_ERROR, ...)`) e imprime um aviso na consola;
- cada pedido deve ser registado na consola com os valores medidos.

Tabela 7: Ligações entre o sensor DHT20 e o ESP32-C6 DevKit.

Pino do DHT20	Função	Ligação ao ESP32-C6 DevKit
1	VDD	`3V3` (+ condensador 100 nF para `GND`)
2	SDA	`GPIO 6` (+ *pull-up* 4,7 kΩ)
3	GND	`GND`
4	SCL	`GPIO 7` (+ *pull-up* 4,7 kΩ)

Este exemplo estabelece um servidor web no ESP32-C6 que opera como cliente da rede Wi-Fi para obter um endereço IP e ter conectividade. Isso implica que quem quiser aceder ao servidor web tem de "descobrir" o endereço IP atribuído ao ESP32-C6, o que nem sempre é fácil. Uma forma de contornar este problema é fazer com que o próprio servidor web desempenhe o papel de *Access Point*: em modo SoftAP, o seu endereço é fixo (`192.168.4.1`).

QUESTÃO 10: Altere o programa da Questão 9 de modo a que o ESP32-C6 funcione **apenas em modo SoftAP** (`WIFI_MODE_AP`), sem se ligar ao telemóvel, permitindo a ligação de um cliente Wi-Fi, atribuindo-lhe automaticamente um endereço IP e servindo-lhe, em `https://192.168.4.1/`, a página com a temperatura e a humidade instantâneas. Requisitos:

- substitua a chamada a `example_connect()` pela inicialização do SoftAP, com base no código do exemplo da Questão 4, e remova o registo dos eventos de STA (`IP_EVENT_STA_GOT_IP` e `WIFI_EVENT_STA_DISCONNECTED`), que deixam de ocorrer;
- inicie o servidor HTTPS quando o SoftAP arrancar (evento `WIFI_EVENT_AP_START`);
- imprima na consola o endereço IP atribuído a cada cliente, tratando o evento `IP_EVENT_ASSIGNED_IP_TO_CLIENT`;
- gere um novo certificado para o servidor com o endereço `192.168.4.1` na extensão SAN, com o comando indicado em seguida, e substitua os ficheiros `main/certs/servercert.pem` e `main/certs/prvtkey.pem` pelos ficheiros gerados.

Depois de instalar o novo certificado, aceda à página e verifique que o *browser* continua a apresentar um aviso. Explique porquê e indique que aviso desapareceu face à Questão 8.

O comando seguinte, adaptado do `README.md` do exemplo, gera uma chave privada RSA de 2048 bits e um certificado *self-signed* válido por 10 anos, com o endereço `192.168.4.1` na extensão SAN (é necessária a ferramenta `openssl`, disponível no Linux e no macOS e instalada com o Git para Windows):

```
openssl req -newkey rsa:2048 -nodes -keyout prvtkey.pem -x509 -days 3650 \
  -out servercert.pem -subj "/CN=ESP32 HTTPS server example" \
```

```
  -addext "keyUsage=critical,digitalSignature,keyEncipherment" \
  -addext "subjectAltName=IP:192.168.4.1"
```

4 Resolução de problemas

Sintoma	Causa provável	Ação
O ESP32-C6 não encontra a rede do telemóvel	*Hotspot* a emitir apenas em 5 GHz	Ativar a banda de 2,4 GHz (no iPhone, **Maximizar compatibilidade**)
`connect to the AP fail` repetido, sem obter IP	*Password* errada, ou segurança da rede inferior ao limiar `threshold.authmode`	Confirmar o SSID e a *password* no `menuconfig`; usar WPA2 ou WPA2/WPA3 no *hotspot*
A STA deixa de tentar ligar-se após uma falha prolongada do AP	Esgotado o número de tentativas (`Maximum retry`)	Reiniciar a placa, ou alterar a lógica de reconexão (Questão 3)
O portátil não se consegue ligar ao SoftAP	Rede SoftAP apenas WPA3 e cliente sem suporte de WPA3	Desativar `Enable WPA3 Personal(SAE) SoftAP` no `menuconfig`
O programa SoftAP termina com erro em `esp_wifi_set_config` (`ESP_ERR_WIFI_PASSWORD`)	*Password* inválida, por exemplo com menos de 8 caracteres	Usar uma *password* com 8 a 63 caracteres
O `ping` ao ESP32-C6 falha	PC noutra rede, ou AP que isola os clientes	Ligar o PC à mesma rede; usar outro AP
O *browser* não abre a página e não mostra aviso que se possa aceitar	Servidor a exigir certificado de cliente	Ativar `Example Configuration > Skip client certificate`
O *browser* não obtém resposta	Acesso por `http://` (porta 80)	Usar `https://<IP>/`
Aviso de segurança no *browser*	Certificado não emitido por CA reconhecida, ou endereço ausente da extensão SAN	Aceitar a exceção; gerar certificado com o IP na SAN (Questão 10)
O exemplo HTTPS fica em `Waiting for IP(s)`	Por omissão, `example_connect()` espera também por um endereço IPv6; a ajuda da opção recomenda desativá-la em redes sem suporte de IPv6	Desativar `Obtain IPv6 address` em `Example Connection Configuration`

5 Acknowledgements

Originally authored by Paulo C. Bartolomeu